

JavaScript

DOM & Events

Week 4 · Friday — Web Programming

Make web pages ALIVE with code

JS

01

What is the DOM?

The tree that represents your web page

02

Selecting Elements

querySelector / getElementById

03

Changing Content & Style

innerHTML,.textContent, style

04

Event Listeners

click, input, keydown, submit

05

Creating & Removing Nodes

createElement, appendChild, remove

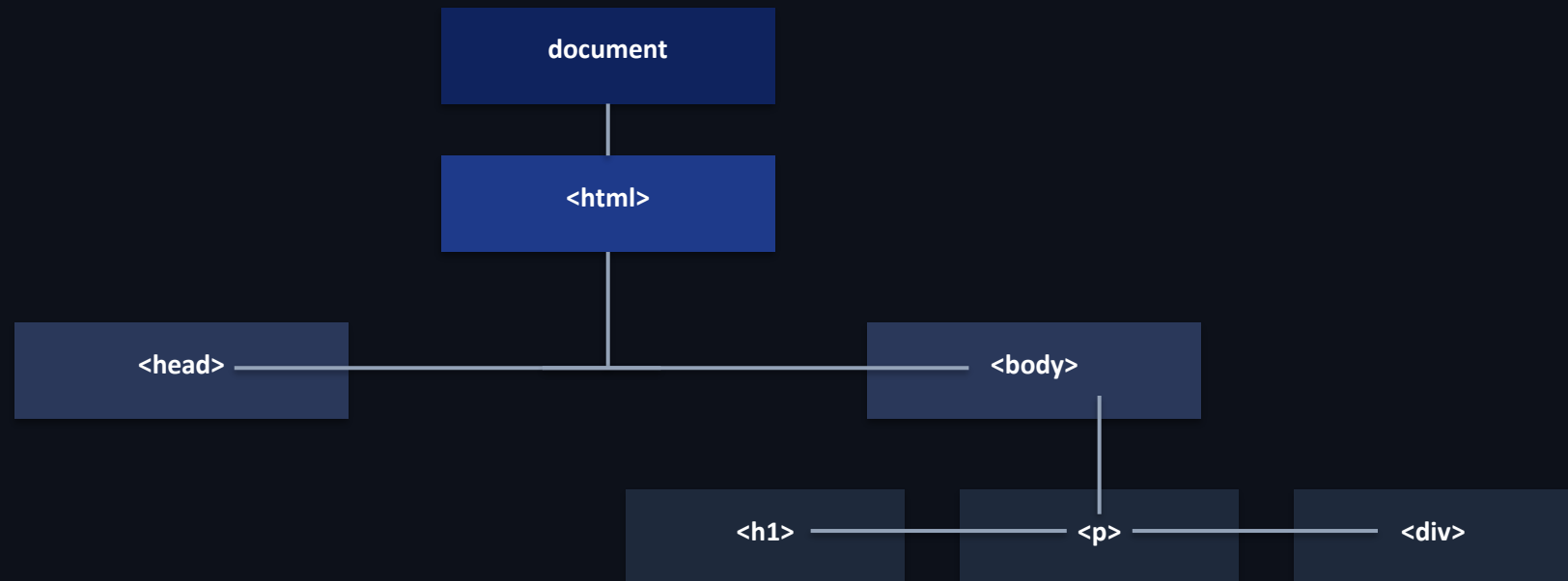
06

Career Corner

Real job skill – every front-end role needs this

DOM = Document Object Model

When a browser loads an HTML file, it converts every tag into a JavaScript object and arranges them in a tree structure — that tree is the DOM. JavaScript can read, change, add, or delete any node in the tree, and the browser immediately updates what the user sees.



JavaScript can touch ANY node in this tree!

How the browser converts your code into a living page

1 You write HTML

```
<body>
  <h1>Hello</h1>
  <p>World</p>
</body>
```



2 Browser parses

```
document
├─ body
│   ├─ h1 : "Hello"
│   └─ p  : "World"
```



3 JS can change it

```
document
  .querySelector("h1")
  .textContent = "Hi!"
```

The DOM is live — change it with JS, the screen updates instantly (no page reload!)

QUICK CHECK

What does 'DOM' stand for?

- A Document Object Model
- B Data Object Map
- C Dynamic Output Manager
- D Display Object Module

ANSWER REVEALED

What does 'DOM' stand for?

✓ Document Object Model

B Data Object Map

C Dynamic Output Manager

D Display Object Module

Explanation:

DOM = Document Object Model. The browser converts HTML into a tree of objects (nodes) that JavaScript can read and modify at any time.

Before you can change anything, you must FIND it in the DOM

getElementById()

Find by id attribute – returns one element

```
const btn = document.getElementById("myBtn");
```

querySelector()

CSS selector syntax – returns FIRST match

```
const h1 = document.querySelector("h1");
```

querySelectorAll()

Returns ALL matches as a NodeList

```
const lis = document.querySelectorAll("ul li");
```

Pro tip: `querySelector()` uses the same selectors as CSS — if you know CSS selectors, you already know how to select DOM nodes!

Selector Cheat-Sheet (Visual)

WEEK 4 · FRIDAY

Same CSS selector rules work inside `querySelector()`

#myId

Select by ID

```
document.querySelector('#myId')
```

.myClass

Select by class

```
document.querySelector('.myClass')
```

div

Select by tag name

```
document.querySelectorAll('div')
```

div > p

Direct child

```
document.querySelectorAll('div > p')
```

[type=text]

Select by attribute

```
document.querySelector('[type=text]')
```

:first-child

First child

```
document.querySelector('li:first-child')
```


QUICK CHECK

Which method returns ALL elements that match a CSS selector?

A `document.getElementById()`

B `document.querySelector()`

C `document.querySelectorAll()`

D `document.getElement()`

ANSWER REVEALED

Which method returns ALL elements that match a CSS selector?

A `document.getElementById()`

B `document.querySelector()`

✓ `document.querySelectorAll()`

D `document.getElement()`

Explanation:

`querySelectorAll()` returns a `NodeList` — like an array — of every element matching the selector. `querySelector()` returns only the FIRST match. `getElement()` doesn't exist.

Once selected, you can READ or WRITE any property

.textContent

Plain text only — safe (ignores HTML tags)

```
el.textContent = "Hello World";  
console.log(el.textContent); // Hello World
```

.innerHTML

HTML markup — powerful but be careful with user input!

```
el.innerHTML = "<strong>Bold text</strong>";  
el.innerHTML += "<li>New item</li>";
```

.style

Inline CSS via JavaScript — camelCase property names

```
el.style.color      = "red";  
el.style.fontSize  = "24px";  
el.style.display    = "none";
```

Changing Classes with classList

WEEK 4 · FRIDAY

Better than .style — add/remove CSS classes instead of inline styles

JS

```
const card = document.querySelector('.card');

card.classList.add('highlight');      // add a class
card.classList.remove('hidden');      // remove a class
card.classList.toggle('active');      // add if absent, remove if present
card.classList.contains('active');    // → true or false

// Example: dark-mode toggle button
document.body.classList.toggle('dark-mode');
```

Why classList?

Separates JS behaviour from CSS appearance

Your CSS file controls .highlight, .dark-mode etc.

JS only switches class names

This is the professional pattern used in every modern framework

Career tip: React, Vue, Angular all use classList patterns internally. Learn it once, use it everywhere.

QUICK CHECK

A student wrote: `el.innerHTML = userInput;` Why is this dangerous?

- A `innerHTML` is too slow for user input
- B User input could contain HTML/JS that executes (XSS attack)
- C `innerHTML` only works with numbers
- D `el.innerHTML` does not exist — use `el.text` instead

ANSWER REVEALED

A student wrote: `el.innerHTML = userInput;` Why is this dangerous?

- A `innerHTML` is too slow for user input
- ✓ User input could contain HTML/JS that executes (XSS attack)
- C `innerHTML` only works with numbers
- D `el.innerHTML` does not exist — use `el.text` instead

Explanation:

XSS (Cross-Site Scripting): if `userInput` contains `<script>stealCookies()</script>`, `innerHTML` runs it. Use `textContent` for any user-provided text — it treats the input as plain text, not markup.

Events = things that happen in the browser (click, keypress, form submit...)

```
JS
// Syntax:
element.addEventListener(eventType, callbackFunction);

// Example 1 - click button to change text
const btn = document.querySelector('#myBtn');
btn.addEventListener('click', function() {
  document.querySelector('h1').textContent = 'Button clicked!';
});

// Example 2 - arrow function (modern style)
btn.addEventListener('click', () => {
  btn.classList.toggle('active');
});
```

Common Events

click	Mouse button pressed and released
dblclick	Double-click
mouseover	Cursor enters element
keydown	Keyboard key pressed
input	Value changes in input field
submit	Form submitted
load	Page fully loaded
DOMContentLoaded	HTML parsed (faster than load)

Every listener receives an event object — packed with useful information

```
JS
// The callback receives an event object (usually named 'e' or 'event')
btn.addEventListener('click', (e) => {
  console.log(e.type);           // 'click'
  console.log(e.target);         // the element that was clicked
  console.log(e.target.id);      // its id attribute
});

// Keyboard events
document.addEventListener('keydown', (e) => {
  console.log(e.key);            // 'Enter', 'a', 'ArrowUp' ...
  if (e.key === 'Enter') sendMessage();
});

// Prevent default browser action (e.g. stop form reload)
form.addEventListener('submit', (e) => {
  e.preventDefault();
  validateForm();
});
```

Key Properties

`e.type`

Event name ('click', 'keydown'...)

`e.target`

Element that triggered the event

`e.key`

Key name for keyboard events

`e.preventDefault()`

Stop the default browser behaviour

`e.stopPropagation()`

Stop event from bubbling up

Event Bubbling: events travel from the target up to the document root

Bubbling (default)

Event fires on <button>
then bubbles up to <div>
then to <body>
then to document
Use `e.stopPropagation()` to stop

document

<body>

↑ bubble

<div>

↑ bubble

<button>

↑ bubble

Capture phase

Optional — event
travels DOWN first
Pass `{capture:true}`
as 3rd argument to
`addEventListener()`

JS

```
// Listen on parent for ALL child clicks
ul.addEventListener('click', (e) => {
  if (e.target.tagName === 'LI') {
    e.target.classList.toggle('done');
  }
});
// ↑ Event Delegation pattern
```

QUICK CHECK

You have a form. When the user clicks Submit, the page reloads and you lose the data. Which line fixes this?

A `form.removeEventListener('submit');`

B `e.stopPropagation();`

C `e.preventDefault();`

D `form.addEventListener('cancel', handler);`

ANSWER REVEALED

You have a form. When the user clicks Submit, the page reloads and you lose the data. Which line fixes this?

A `form.removeEventListener('submit');`

B `e.stopPropagation();`

✓ `e.preventDefault();`

D `form.addEventListener('cancel', handler);`

Explanation:

`e.preventDefault()` cancels the browser's default action. For a form, that default is to submit (causing a page reload). After calling it, your JavaScript handles the data instead.

JavaScript can build and destroy DOM nodes at runtime

```
JS

// — CREATE —————
const li = document.createElement('li'); // new <li> node
li.textContent = 'Buy groceries';        // set its text
li.classList.add('todo-item');           // add a class

const ul = document.querySelector('#todo-list');
ul.appendChild(li);                      // add to the page

// — INSERT AT SPECIFIC POSITION —————
ul.insertBefore(li, ul.firstChild);      // prepend
ul.insertAdjacentHTML('beforeend', '<li>More</li>');

// — REMOVE —————
li.remove();                             // remove itself
ul.removeChild(li);                      // parent removes child
```

The 3-Step Pattern

Step 1

`createElement(tag)`

Create a new node in memory

Step 2

`Set properties`

`textContent`, `classList`, `id`...

Step 3

`appendChild()`

Attach it to the live DOM

Mini Project: Todo List

WEEK 4 · FRIDAY

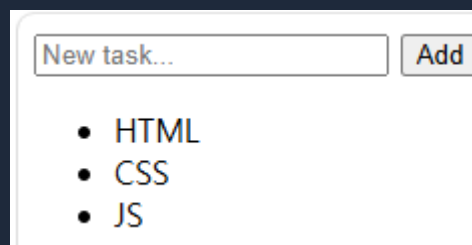
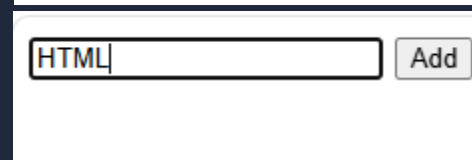
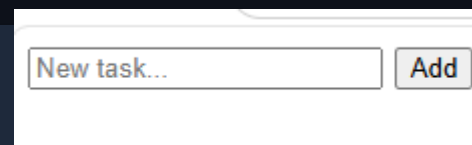
Combining everything: select → listen → create → remove

JS

```
<!DOCTYPE html>
<body>
  <input id='inp' placeholder='New task...'/>
  <button id='addBtn'>Add</button>
  <ul id='list'></ul>

  <script>
    const btn = document.querySelector('#addBtn');
    const inp = document.querySelector('#inp');
    const list = document.querySelector('#list');

    btn.addEventListener('click', () => {
      if (!inp.value.trim()) return; // guard empty
      const li = document.createElement('li');
      li.textContent = inp.value;
      li.addEventListener('click', () => li.remove()); // click to delete
      list.appendChild(li);
      inp.value = ''; // clear input
    });
  </script>
</body>
```



Skills used:

- `querySelector` x3
- `addEventListener` x2
- `createElement`
- `textContent`
- `appendChild`
- `remove()`
- Guard clause (if !value)
- Inline event on new node

QUICK CHECK

What is the correct order to add a new `<li>` to a `<ul>`?

A `appendChild` first, then `createElement`, then `textContent`

B `createElement` → `set textContent` → `appendChild`

C `innerHTML` → `createElement` → `remove`

D `getElementById` → `createElement` → `insertBefore`

ANSWER REVEALED

What is the correct order to add a new `<li>` to a `<ul>`?

A `appendChild` first, then `createElement`, then `textContent`

✓ `createElement` → `set textContent` → `appendChild`

C `innerHTML` → `createElement` → `remove`

D `getElementById` → `createElement` → `insertBefore`

Explanation:

Always: (1) Create the node in memory, (2) Configure it (text, classes, events), (3) Append it to the DOM. Appending first then setting properties also works, but attaching an empty or misconfigured node can cause a visible flash.

Next week: loading REAL data from the internet and injecting it into the DOM

JS

```
// Fetch data from an API, then show it in the DOM
fetch('https://jsonplaceholder.typicode.com/posts/1')
  .then(res => res.json())
  .then(data => {
    document.querySelector('#title').textContent = data.title;
    document.querySelector('#body').textContent = data.body;
  })
  .catch(err => console.error('Error:', err));

// Everything you learned today is used here:
// querySelector → textContent → dynamic update
```

What this unlocks

Display weather data
Show product listings
Live search results
Social media feeds
Game leaderboards
ALL use DOM + Fetch

This is the pattern behind every modern web application. Master today's DOM skills and Fetch is just one more line.

Every front-end interview asks about the DOM

Kakao

Virtual DOM, Event delegation, SPA routing

Naver

Search autocomplete, Live news feeds

Samsung

SmartTV UI – pure DOM manipulation

Coupang

Product listing filters, cart updates

Krafton

PUBG Web – game lobby, matchmaking UI

Common Interview Questions

- How would you implement event delegation?
- What is the difference between innerHTML and textContent?
- How do you prevent a form from reloading the page?
- Explain event bubbling and capturing.

Homework: Interactive Color Palette

Required (Grade 7/10)

- Create 6 colored div boxes
- Click a box → it expands (classList.toggle + CSS transition)
- Show the hex color code when expanded (textContent)

Challenge (Grade 9/10)

- Add an 'Add Color' button + text input
- User enters a hex color, new box appears
- Double-click any box to remove it (dblclick event)

Extra Credit (10/10)

- Save the color list to localStorage
- Restore boxes on page reload
- Export palette as a PNG (Bonus – research on your own)

[Submit GitHub Pages link via the Homework Submission Form](#) (Week 4 Friday, Assignment Type: DOM Practice)

DOM

Browser converts HTML into a tree of objects JS can touch

Select

querySelector / querySelectorAll — CSS selectors work!

Read & Write

textContent (safe), innerHTML (careful), classList (clean)

Listen

addEventListener(type, callback) — one method to rule them all

Event Object

e.target, e.key, e.preventDefault() — data inside every event

Create / Remove

createElement → configure → appendChild; .remove() to delete

Next week: Fetch API — load real data, combine with DOM manipulation to build live web apps.